

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

Тема: «Взаимодействие с внешним API»

Выполнил:

Студент группы J3212
Карташов Игорь Олегович

Преподаватель:

2026 г.

Цель работы

Привязать ранее реализованный статический сайт DataFlow к внешнему API средствами функции `fetch`. Реализовать моковое (тестовое) API с помощью библиотеки JSON Server и подключить систему авторизации пользователей.

Ход работы

1. Создание моковой базы данных

Создан файл `db.json`, содержащий 6 основных сущностей системы:

- **users** — пользователи системы (email, пароль, имя, роль, команда);
- **pipelines** — список пайплайнов (название, cron-расписание, статус, время последнего запуска);
- **tasks** — задачи в DAG-графе каждого пайплайна (название, иконка, порядок, время выполнения);
- **logs** — строки логов пайплайнов (время, уровень, сообщение);
- **runs** — история запусков (идентификатор, время, длительность, статус);
- **connections** — подключения к внешним сервисам (тип, хост, статус).

JSON Server автоматически предоставляет REST API по данным из этого файла. Основные используемые эндпоинты:

- GET `/pipelines` — список всех пайплайнов;
- GET `/pipelines/:id` — получение конкретного пайплайна;
- GET `/tasks?pipelineId=1&_sort=order` — задачи пайплайна, отсортированные по порядку выполнения;
- POST `/users` — регистрация нового пользователя.

2. Обёртка над `fetch` (`js/api.js`)

Для упрощения сетевых запросов создан модуль, содержащий базовый URL (`API_BASE = 'http://'`) и две универсальные функции:

- `api(endpoint)` — выполняет GET-запрос к JSON Server, обрабатывает ошибки HTTP и возвращает распарсенный JSON;
- `apiPost(endpoint, data)` — выполняет POST-запрос, передавая данные в формате JSON в теле запроса.

3. Авторизация (js/auth.js)

Реализована полноценная логика клиентской авторизации:

- `handleLogin(event)` — обработчик формы входа. Выполняет `GET /users?email=...`, сравнивает пароль. При успехе сохраняет `userId` в `localStorage` и перенаправляет пользователя на `index.html`.
- `handleRegister(event)` — проверяет совпадение паролей и уникальность email, после чего выполняет `POST /users` для создания учётной записи.
- `requireAuth()` — функция защиты маршрутов. Проверяет наличие `userId` в `localStorage`, при отсутствии — принудительно перенаправляет на `login.html`. Вызывается на всех внутренних страницах.
- `logout()` — очищает `localStorage` и возвращает на страницу входа.

Ошибки авторизации (неверный пароль, существующий email, недоступный сервер) выводятся в специально подготовленный блок `<div id="authError">` над формой.

4. Динамическая загрузка данных

Страница `index.html` (список пайплайнов):

Элемент `<tbody>` в HTML изначально пуст. При загрузке страницы выполняется `fetch GET /pipeline`. На основе полученного массива JavaScript динамически генерирует строки таблицы (`<tr>`) с заполнением всех колонок и бейджей статуса. Поиск и фильтрация по статусу корректно работают поверх сгенерированных элементов.

Страница `pipeline-detail.html` (детали пайплайна):

Идентификатор пайплайна извлекается из URL-параметра (например, `?id=1`). При загрузке страницы выполняются 4 параллельных запроса с использованием `Promise.all`:

1. `GET /pipelines/:id` — для вывода метаданных (название, статус);
2. `GET /tasks?pipelineId=:id&_sort=order` — для построения DAG-графа;
3. `GET /logs?pipelineId=:id` — для заполнения блока логов;
4. `GET /runs?pipelineId=:id` — для таблицы истории запусков.

DAG-граф строится программно: задачи группируются по полю `order`, параллельные задачи (с одинаковым `order`) объединяются в flex-контейнеры для отображения в один ряд.

Страница `profile.html` (профиль пользователя):

Осуществляется загрузка данных текущего пользователя, списка его пайплайнов и истории запусков для агрегации статистики (подсчёт количества DAGs, общее число запусков, процент успешных выполнений), а также загружается список подключений (`GET /connections?us`).

Структура файлов (JavaScript)

js/

api.js	- обёртка fetch (GET, POST)
auth.js	- авторизация и управление сессией
index.js	- загрузка и рендер таблицы пайплайнов
pipeline-detail.js	- загрузка и рендер графа, логов, истории
profile.js	- загрузка и рендер профиля, статистики, connections
theme.js	- переключатель тем (из HW3)

Порядок запуска проекта

1. Запуск JSON Server: `npx json-server db.json --port 3000`
2. Запуск локального HTTP-сервера: `python -m http.server 8080`
3. Открыть веб-браузер по адресу: `http://localhost:8080/login.html`
4. Использовать тестовые данные: логин `igor@company.com`, пароль `admin123`.

Вывод

В ходе выполнения лабораторной работы статический сайт DataFlow был успешно интегрирован с моковым REST API на базе JSON Server. Все информационные блоки (список пайплайнов, графы задач, терминалы логов, история запусков, данные профиля и подключения) теперь загружаются динамически посредством асинхронных **fetch**-запросов.

Реализована надежная клиентская авторизация: процесс регистрации создает новую запись через **POST /users**, процесс входа валидирует данные через **GET /users**, а состояние сессии сохраняется в **localStorage**. Внедрена защита маршрутов — закрытые страницы интерфейса автоматически проверяют наличие активной сессии и перенаправляют неавторизованных посетителей на экран входа.

Приложение. Результаты работы

```
Press CTRL-C to stop
Watching db.json...

( ͡° ͜ʖ ͡° )

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/users
http://localhost:3000/pipelines
http://localhost:3000/tasks
http://localhost:3000/logs
http://localhost:3000/runs
http://localhost:3000/connections
█
```

Рис. 1: Запущенный JSON Server и логи обработки REST-запросов

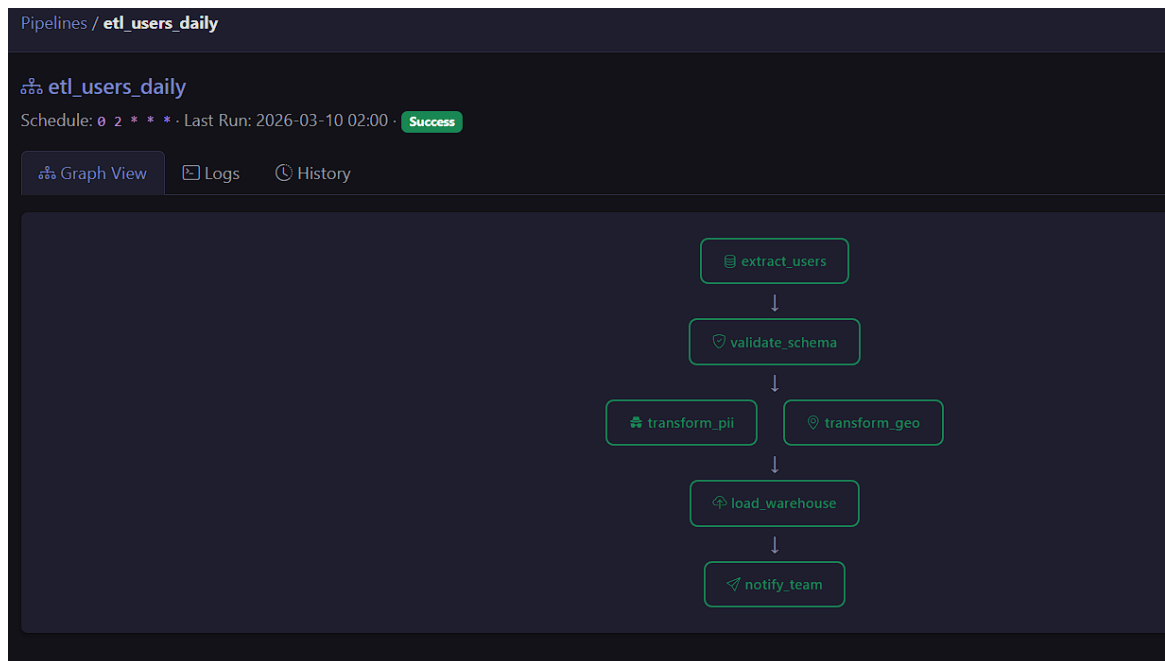


Рис. 2: Детальная страница: программно построенный DAG-граф из БД

DataFlow | Pipelines | Search pipelines... | **All** | Success | Failed | Running | Profile

Name	Schedule	Last Run	Status	Actions
etl_users_daily	0 2 * * *	2026-03-10 02:00	Success	▶ ⏸
sync_orders_warehouse	* / 30 * * * *	2026-03-10 11:30	Success	▶ ⏸
ml_training_pipeline	0 6 * * 1	2026-03-09 06:00	Running	▶ ⏸
report_generator_v2	0 8 * * *	2026-03-10 08:00	Failed	▶ ⏸
clickstream_ingestion	* / 5 * * * *	2026-03-10 11:45	Success	▶ ⏸
backup_postgres_daily	0 1 * * *	2026-03-10 01:00	Success	▶ ⏸

Рис. 3: Главная страница: таблица пайплайнов загружена через `fetch`

```

1  {
2    "users": [
3      {
4        "id": 1,
5        "firstName": "Igor",
6        "lastName": "Kartashov",
7        "email": "igor@company.com",
8        "password": "admin123",
9        "role": "Admin",
10       "team": "Data Engineering",
11       "timezone": "UTC+3",
12       "memberSince": "Jan 2025"
13     }
14   ],
15   "pipelines": [
16     { "id": 1, "name": "etl_users_daily", "schedule": "0 2 * * *", "lastRun": "2026-03-10 02:00"
17     { "id": 2, "name": "sync_orders_warehouse", "schedule": "*/30 * * * *", "lastRun": "2026-03-
18     { "id": 3, "name": "ml_training_pipeline", "schedule": "0 6 * * 1", "lastRun": "2026-03-09 6
19     { "id": 4, "name": "report_generator_v2", "schedule": "0 8 * * *", "lastRun": "2026-03-10 08
20     { "id": 5, "name": "clickstream_ingestion", "schedule": "*/5 * * * *", "lastRun": "2026-03-1
21     { "id": 6, "name": "backup_postgres_daily", "schedule": "0 1 * * *", "lastRun": "2026-03-10
22   ],
23   "tasks": [
24     { "id": 1, "pipelineId": 1, "taskId": "task_001", "name": "extract_users", "icon": "database
25     { "id": 2, "pipelineId": 1, "taskId": "task_002", "name": "validate_schema", "icon": "shield
26     { "id": 3, "pipelineId": 1, "taskId": "task_003", "name": "transform_pii", "icon": "incognit
27     { "id": 4, "pipelineId": 1, "taskId": "task_004", "name": "transform_geo", "icon": "geo-alt"
28     { "id": 5, "pipelineId": 1, "taskId": "task_005", "name": "load_warehouse", "icon": "cloud-u
29     { "id": 6, "pipelineId": 1, "taskId": "task_006", "name": "notify_team", "icon": "send", "st
30   ],
31   "logs": [
32     { "id": 1, "pipelineId": 1, "time": "2026-03-10 02:00:01", "level": "info", "message": "Star
33     { "id": 2, "pipelineId": 1, "time": "2026-03-10 02:00:01", "level": "info", "message": "Exec
34     { "id": 3, "pipelineId": 1, "time": "2026-03-10 02:00:15", "level": "ok", "message": "Extrac
35     { "id": 4, "pipelineId": 1, "time": "2026-03-10 02:00:16", "level": "info", "message": "Exec
36     { "id": 5, "pipelineId": 1, "time": "2026-03-10 02:00:18", "level": "warn", "message": "Colu
37     { "id": 6, "pipelineId": 1, "time": "2026-03-10 02:00:19", "level": "info", "message": "Runn
38     { "id": 7, "pipelineId": 1, "time": "2026-03-10 02:00:38", "level": "ok", "message": "Transf
39     { "id": 8, "pipelineId": 1, "time": "2026-03-10 02:00:53", "level": "info", "message": "Exec
40     { "id": 9, "pipelineId": 1, "time": "2026-03-10 02:03:41", "level": "ok", "message": "Loaded
41     { "id": 10, "pipelineId": 1, "time": "2026-03-10 02:03:43", "level": "ok", "message": "DAG c

```

Рис. 4: Пример из БД